САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Попов Никита

Группа К3344

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и мой)

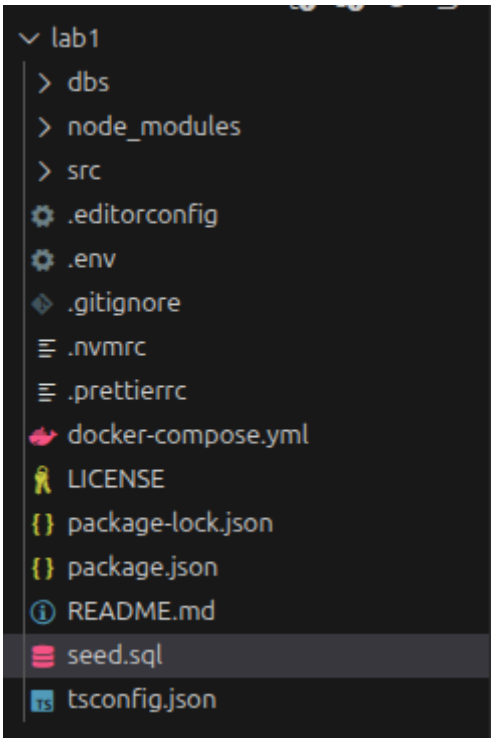
С учётом имеющихся данных по результатам ДЗ1 и ДЗ2 реализуйте:

- модели
- представления
- контроллеры

В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Ход работы

Скопируем бойлерплейт. Получим следующую структуру проекта:



Настроим docker-compose.yml файл, чтобы у нас правильно запускался контейнер с postgres:

lab1 > docker-compose.yml

```yaml
services:
  db:
    image: postgres
    restart: always
    ports:
      - 15432:5432
    volumes:
      - ./dbs/postgres-data:/var/lib/postgresql
    env_file:
      - .env
```

Напишем модели для наших сущностей, которые будут храниться в бд. Например, сущность ресторана:

```
export enum PriceLevel {
    LOW = 'LOW',
    MEDIUM = 'MEDIUM',
    MEDIUM_HIGH = 'MEDIUM_HIGH',
    HIGH = 'HIGH',
}

@Entity()
export class Restaurant extends BaseEntity {
    @PrimaryGeneratedColumn('uuid')
    id: string;

    @Column({ type: 'varchar', length: 300 })
    name: string;

    @Column({ type: 'text', nullable: true })
    description?: string;

    @Column({ type: 'varchar', length: 300 })
    address: string;

    @Column({ type: 'varchar', length: 150 })
    city: string;

    @Column({
        type: 'enum',
        enum: PriceLevel,
    })
    priceLevel: PriceLevel;

    @Column({ type: 'int' })
    capacity: number;

    @Column({ type: 'float', nullable: true })
    rating?: number;

    @CreateDateColumn()
    createdAt: Date;

    @OneToMany(() => MenuItem, (item) => item.restaurant)
    menuItems: MenuItem[];

    @OneToMany(() => Booking, (booking) => booking.restaurant)
    bookings: Booking[];

    @OneToMany(() => Review, (review) => review.restaurant)
    reviews: Review[];
}
```

Следующим этапом напишем контроллер, в котором сразу опишем и представление в виде json ответа. Например, метод GET контроллера для ресторана:

```
@JsonController("/restaurants")
export class RestaurantsController extends BaseController {
    private restaurantRepo = dataSource.getRepository(Restaurant);

    @Get()
    @OpenAPI({ summary: 'Получение списка ресторанов' })
    @ResponseSchema(ListRestaurantsResponseDto, { statusCode: 200 })
    async list(
        @QueryParams({ type: ListRestaurantsQuery })
        query: ListRestaurantsQuery,
    ): Promise<ListRestaurantsResponseDto> {
        const page = query.page ?? 1;
        const pageSize = query.pageSize ?? 20;

        const qb = this.restaurantRepo.createQueryBuilder('restaurant');

        if (query.city) {
            qb.andWhere('restaurant.city ILIKE :city', {
                city: `%${query.city}%`,
            });
        }

        if (query.priceLevel) {
            qb.andWhere('restaurant.priceLevel = :priceLevel', {
                priceLevel: query.priceLevel,
            });
        }

        if (query.name) {
            qb.andWhere('restaurant.name ILIKE :name', {
                name: `%${query.name}%`,
            });
        }

        if (query.sortBy) {
            qb.orderBy(
                `restaurant.${query.sortBy}`,
                (query.sortOrder || 'ASC') as 'ASC' | 'DESC',
            );
        }

        qb.skip((page - 1) * pageSize).take(pageSize);

        const [items, total] = await qb.getManyAndCount();

        return {
            data: {
                items,
                total,
                page,
```

Представления, используемые в этом методе:

```
class PaginatedRestaurantsDto {
    @Type(() => Restaurant)
    items: Restaurant[];

    total: number;
    page: number;
    pageSize: number;
}

class ListRestaurantsResponseDto {
    data: PaginatedRestaurantsDto;
}
```

В итоге получаем работоспособный метод GET без ограничении на роль, который возвращает список ресторанов с пагинацией, при этом имеется фильтрация по различным параметрам:

GET /api/v1/restaurants Получение списка ресторанов

Parameters

Name	Description
city **string** *(query)*	city
cuisineId **string** *(query)*	cuisineId
priceLevel **string** *(query)*	--
name **string** *(query)*	name
sortBy **string** *(query)*	sortBy
sortOrder **string** *(query)*	sortOrder
page **integer** *(query)*	page
pageSize **integer** *(query)*	pageSize

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/v1/restaurants?city=%D0%9C%D0%BE%D1%81%D0%BA%D0%B2%D0%B0&priceLevel=LO
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/api/v1/restaurants?city=%D0%9C%D0%BE%D1%81%D0%BA%D0%B2%D0%B0&priceLevel=LOW
```

Server response

Code	Details
200	Response body


```
{
  "data": {
    "items": [
      {
        "id": "9a81be79-a6e7-410c-8548-4c733378e594",
        "name": "Быстрое питание",
        "description": "Бургеры, картофель фри",
        "address": "пр. Победы, 5",
        "city": "Москва",
        "priceLevel": "LOW",
        "capacity": 80,
        "rating": 3.9,
        "createdAt": "2026-03-07T12:49:28.424Z"
      },
      {
        "id": "6a90e8f3-efa2-4fe4-9d82-99bd02162b08",
        "name": "Кофейня",
        "description": "Кофе, десерты",
        "address": "ул. Кофейная, 3",
        "city": "Москва",
        "priceLevel": "LOW",
        "capacity": 20,
        "rating": 4.3,
        "createdAt": "2026-03-07T12:49:28.424Z"
      },
      {
```


Response headers

```
access-control-allow-origin: *
connection: keep-alive
content-length: 1141
content-type: application/json; charset=utf-8
date: Tue,10 Mar 2026 16:39:27 GMT
etag: W/"475-lY7ImycxKV6md3LKO4dCVD1dxrE"
```

Вывод

В итоге, мы получили полноценное rest api приложения для бронирования мест в ресторане. Были рассмотрены все возможные ошибки и случаи взаимодействия приложения с пользователем.